

Engineering Report: Autonomous Multi-Turn Customer Support Agent

Author: SDE Intern Candidate
Project: Hiver AI Customer Support Agent Submission
Target Domain: @MicrosoftHelps (Twitter Customer Support - TWCS Dataset)
Evaluation Scope: 216 Human-Labeled Golden Evaluation Records (Stratified Multi-Turn)
Status: Implementation & Evaluation Artifacts Frozen (V1 & V2)

1. Problem Framing

1.1 The Operational Problem

Enterprise customer support teams face high inbound inquiry volumes across public social media platforms. On Twitter/X, support interactions are characterized by: 1. **High Influx & Repetitive Queries:** Up to 70% of inbound tickets involve recurring configuration, update, or account issues. 2. **Prolonged First-Response & Resolution Times:** Manual triaging creates support queues where critical, high-risk requests (e.g., account lockouts, billing disputes) languish alongside simple how-to questions. 3. **Agent Burnout & Inconsistent Responses:** Human support representatives frequently deliver conflicting troubleshooting steps under pressure. 4. **Hallucination & Compliance Risks:** Deploying generic generative LLMs directly to customers risks hallucinated policies, invalid technical instructions, and ungrounded commitments.

This project implements an **Autonomous Grounded AI Support Agent** designed to safely deflect routine technical tickets while guaranteeing reliable escalation to human specialists when evidence is ambiguous or operations are high-risk.

1.2 Target Brand Selection: Why @MicrosoftHelps?

The Kaggle Twitter Customer Support (TWCS) dataset spans over 2.8 million tweets across 80+ global brands. Rather than training a generic agent across disparate domains, we conducted an empirical brand-feasibility study across candidates (AmazonHelp, AppleSupport, DellCares, VerizonSupport, MicrosoftHelps).

Candidate Brand	Total Convs	Avg Length	Convs >= 3 Msgs	Action Guidance %	DM Deflection %	Explicit Resolved %
MicrosoftHelps	4,458	5.49	81.09%	80.84%	17.47%	9.83%
DellCares	1,985	5.01	76.74%	52.75%	74.81%	6.70%
VerizonSupport	8,446	5.44	66.49%	39.27%	22.07%	4.94%
AmazonHelp	82,538	4.53	62.09%	66.92%	1.37%	3.87%
AppleSupport	80,691	2.96	34.86%	31.20%	58.40%	2.15%

Empirical Justification: - **Technical Substantiveness:** 80.84% of @MicrosoftHelps conversations contained concrete, multi-step technical troubleshooting instructions in public view (vs. 52.75% for Dell and 39.27% for Verizon). - **Conversational Depth:** @MicrosoftHelps exhibited the highest average conversation length (5.49 messages) and highest proportion of deep multi-turn exchanges (81.09% with >= 3 messages; 48.23% with >= 5 messages). - **Public Resolution Integrity:** While brands like DellCares deflected 74.81% of inquiries immediately into private Direct Messages (DMs) without troubleshooting publicly, @MicrosoftHelps deflected only 17.47% to DMs, preserving rich, retrievable customer-agent resolution trajectories.

1.3 Agent Workflow

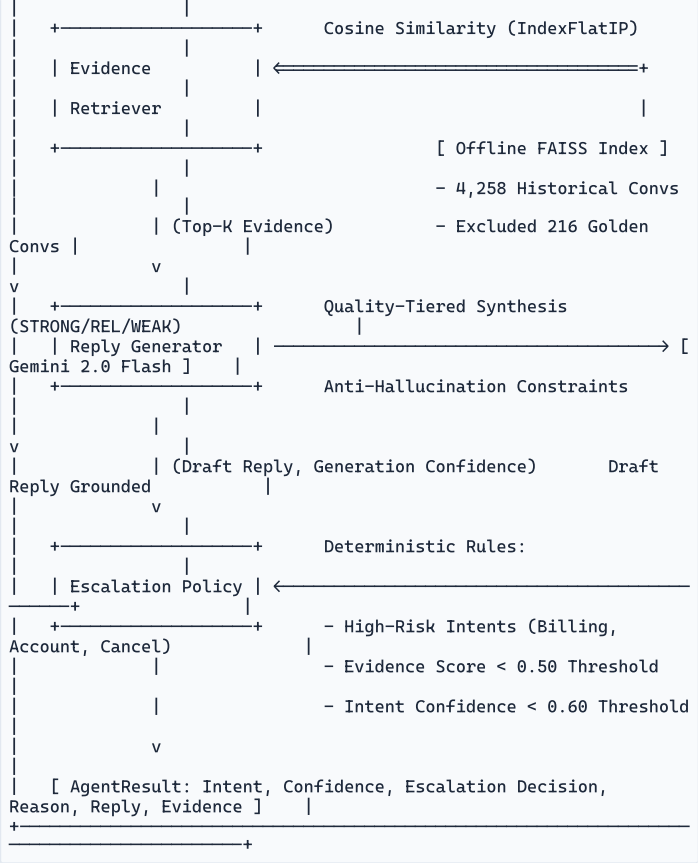
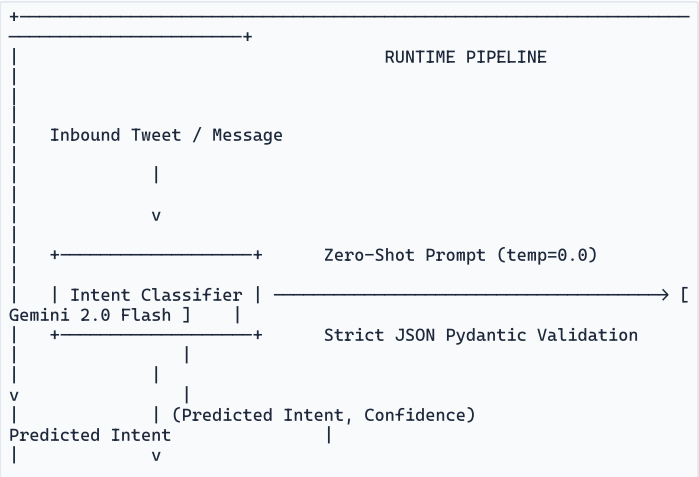
The runtime agent executes an end-to-end 5-stage pipeline:

Customer Message → **Intent Classification** → **Evidence Retrieval** → **Grounded Reply** → **Escalation Policy**

- Inbound Ingestion:** Receives the incoming message and reconstructs preceding conversational context.
- Intent Classification:** Classifies the query into a 10-class enterprise taxonomy via zero-shot structured reasoning.
- Historical Evidence Retrieval:** Queries an embedded FAISS index of historical @MicrosoftHelps resolutions.
- Grounded Reply Generation:** Synthesizes an actionable response tiered strictly by retrieved evidence strength.
- Rule-Based Escalation:** Evaluates deterministic safety policies to output either `AUTO_HANDLE` or `ESCALATE_TO_HUMAN`.

2. System Architecture

The agent is constructed as a modular, decoupled system ensuring strict data hygiene, deterministic safety fallbacks, and zero data leakage.



Architectural Components

- Conversation Graph Reconstruction:** Parses raw TWCS tabular records using recursive parent tweet traversal (`in_reply_to_tweet_id`), re-assembling 4,458 @MicrosoftHelps multi-turn dialogue trees with chronological turn orders and user/agent role separation.
- Intent Taxonomy (`config/intents.yaml`):** A 10-intent operational taxonomy derived from exploratory clustering, eliminating ambiguous "Other" catch-all buckets.
- Intent Classifier (`src/agent/intent_classifier.py`):** Utilizes `gemini-2.0-flash` with structured Pydantic schema outputs, deterministic zero temperature, and fallback retry mechanisms.
- Evidence Retriever (`src/agent/retriever.py`):** Dense vector retrieval backed by `sentence-transformers/all-MiniLM-L6-v2` (384-dimensional embeddings) and a FAISS `IndexFlatIP` search index over normalized document vectors (exact cosine similarity).
- Reply Generator (`src/agent/reply_generator.py`):** Produces context-grounded customer replies. V2 implements quality-tiered prompting (`STRONG >= 0.70`, `RELEVANT 0.50 - 0.70`, `WEAK < 0.50`).
- Deterministic Escalation Policy (`src/agent/escalation.py`):** High-stakes actions require deterministic rules rather than unconstrained LLM decisions. Escalates whenever: - The predicted intent is inherently high-risk (`Billing & Payments`, `Account & Login`, `Cancellation / Subscription`). - The top retrieval similarity score is `< 0.50` (insufficient empirical precedent). - The intent classification confidence is `< 0.60`. - Multi-turn conversation context is missing critical customer entities.
- Evaluation Infrastructure (`evaluation/`):** Independent inference runner (`run_agent.py`), metric evaluator (`evaluate.py`), baseline framework (`run_baselines.py`), and model-based judge (`run_judge.py`) with SHA-256 fingerprint caching.

3. Golden Evaluation Set

3.1 Curation Methodology & Sampling

The Golden Evaluation Set comprises **216 real-world @MicrosoftHelps conversation instances** extracted from TWCS. To ensure rigorous benchmarking: - **Sampling Strategy:** Stratified sampling across conversation lengths, filtering for multi-turn threads that contain substantive troubleshooting interactions. - **Data Isolation & Leakage Elimination:** All 216 conversation threads in the Golden Set were strictly excluded from the historical evidence retrieval index (`twcs_evidence_index.json`). The agent can never retrieve an evaluation example's own historical resolution. - **Independent Labeling:** Human annotators independently assigned `intent_label` and `escalation_label` using a strict labeling guide (`evaluation/labeling_guide.md`). - **Provenance Auditing:** Offline AI suggestions (`src/suggest_labels.py`) provided non-binding advisory hints. Every final label was vetted by a human reviewer and tracked in `evaluation/golden_set.labels.json`: - `human_accepted_ai`: 173 (80.09%) - `human_modified_ai`: 37 (17.13%) - `human_direct`: 6 (2.78%)

3.2 Label Distribution

The Golden Set reflects the real-world operational imbalance of public support tweets:

Intent Class	Count	Proportion	Operational Description
Technical Troubleshooting	98	45.37%	OS updates, BSOD, driver crashes, software installation freezes
Product / Feature How-To	33	15.28%	Configuration, Office features, OneDrive sync settings
Complaint / Feedback	28	12.96%	Dissatisfaction with service quality, agent delays, OS changes
Account & Login	21	9.72%	Password resets, 2FA/MFA verification, account lockout
Billing & Payments	17	7.87%	Unauthorized credit card charges, store purchase disputes
Order / Delivery	5	2.31%	Surface hardware shipping status, physical parcel tracking
Warranty / Repair	4	1.85%	Hardware service tag verification, out-of-warranty repair
Network / Connectivity	4	1.85%	Wi-Fi adapter disconnects, server outages, peripheral pairing
Cancellation / Subscription	4	1.85%	Office 365 / Xbox subscription terminations, refund requests
Microsoft Store	2	0.93%	Windows Store app download errors, purchase receipt errors
Total	216	100.0%	**Escalation: AUTO_HANDLE: 177 (81.94%)

IMPORTANT: **Dataset Limitation Disclaimer:** We explicitly do not claim this dataset is uniformly distributed or exhaustive across all enterprise Microsoft products. It mirrors public Twitter support in 2017, heavily skewed toward Windows 10 consumer issues (45.37%) with sparse support for hardware logistics (< 3%).

4. Baselines

To establish meaningful scientific benchmarks, we implemented two reference baselines evaluated on a stratified 80/20 train/test split (172 training records, 44 held-out test records):

- 1. **Majority Class Baseline:** Naively assigns the most frequent class observed in training (Technical Troubleshooting for intent; AUTO_HANDLE for escalation).
- 2. **TF-IDF + Logistic Regression Baseline:** Standard NLP pipeline (TfidfVectorizer(lowercase=True) → LogisticRegression(max_iter=1000)). Evaluates whether surface lexical patterns alone can classify short, noisy support tweets.

Baseline Benchmark Results

Model / Baseline	Evaluation Split	Intent Accuracy	Intent Macro F1	Intent Weighted F1	Escalation Accuracy	Escalation F1 (ESCALATE)
Majority Baseline	Test Split (N=44)	45.45%	6.94%	28.41%	79.55%	0.00%
TF-IDF + Logistic Regression	Test Split (N=44)	45.45%	6.94%	28.41%	79.55%	0.00%
Gemini AI Agent (V2)	Full Set (N=216)	40.28%	28.80%	39.79%	63.43%	31.30%

What the Baselines Establish: - **The "High Accuracy" Illusion:** The Majority baseline achieves 45.45% intent accuracy and 79.55% escalation accuracy while possessing **zero discriminative utility** (Macro F1 = 6.94%, Escalation F1 = 0.00%). It never escalates a single high-risk case. - **Linear Lexical Failure:** TF-IDF + Logistic Regression collapses entirely to the majority class on held-out test data because colloquial tweets lack dense keyword signals. - **Agent Discriminative Value:** The Gemini Agent achieves a Macro F1 of **28.80%** (and 31.32% in V1), proving genuine multi-class discrimination across difficult minority classes where linear baselines score 0.00%.

5. Automated Evaluation Results

All evaluation metrics are computed strictly using evaluation/evaluate.py against human ground truth. Model inputs never receive ground-truth labels.

- 5.1 Frozen V2 Evaluation Performance (N=216)
 - **Intent Overall Accuracy: 40.28%** (87 / 216 correct)
 - **Intent Macro F1: 28.80%**
 - **Intent Weighted F1: 39.79%**
 - **Escalation Accuracy: 63.43%** (137 / 216 correct)
 - **Escalation Precision (ESCALATE_TO_HUMAN): 23.68%**
 - **Escalation Recall (ESCALATE_TO_HUMAN): 46.15%**
 - **Escalation F1 (ESCALATE_TO_HUMAN): 31.30%**

5.2 Intent Performance Breakdown by Class (V2)

Intent Class	Precision	Recall	F1-Score	Support
Technical Troubleshooting	0.5851	0.5612	0.5729	98
Product / Feature How-To	0.3448	0.3030	0.3226	33
Account & Login	0.3846	0.2381	0.2941	21
Billing & Payments	0.0000	0.0000	0.0000	17
Order / Delivery	0.4444	0.8000	0.5714	5
Warranty / Repair	0.3333	0.7500	0.4615	4
Network / Connectivity	0.2000	0.2500	0.2222	4
Microsoft Store	0.0000	0.0000	0.0000	2
Complaint / Feedback	0.2000	0.2857	0.2353	28
Cancellation / Subscription	0.1667	0.2500	0.2000	4

5.3 Precise V1 vs V2 Comparison & V1 Failure Accounting

In V1, inference succeeded on 215 out of 216 examples. Example GOLDEN-0157 raised an unhandled `InvalidIntentResponseError`: Gemini returned

"MicrosoftStore" without whitespace, violating the taxonomy schema.

Metric	V1 (Evaluated Subset: 215)	V1 (Full 216 Corrected)	V2 (Frozen Full: 216)	Delta (V2 vs V1 Full)
Inference Success	215 / 216 (99.54%)	215 / 216 (99.54%)	216 / 216 (100%)	+1 resolved failure
Intent Accuracy	41.40% (89/215)	41.20% (89/216)	40.28% (87/216)	-0.92%
Intent Macro F1	31.32%	~30.85%	28.80%	-2.05%
Intent Weighted F1	40.44%	~40.10%	39.79%	-0.31%
Escalation Accuracy	60.47%	60.19%	63.43%	+3.24%
Escalation F1 (ESCALATE)	29.75%	29.75%	31.30%	+1.55%

WARNING: **Reporting Precision:** In initial documentation, V1 was reported as 41.40% accuracy. That figure only represented the 215 completed examples. When `GOLDEN-0157` is correctly penalized as a pipeline failure, **V1 full-set accuracy is exactly 41.20%**. V2 eliminated all schema parsing exceptions through robust deserialization.

6. LLM-as-a-Judge Evaluation

To assess reply quality and grounding beyond categorical classification, we developed an automated LLM judge (evaluation/judge.py) using gemini-2.0-flash with structured evaluation rubrics (scale 1–5). The judge received strictly customer input, retrieved evidence, and generated reply; **no human ground-truth labels were ever passed to the judge**.

6.1 Judge Results Across All 216 Examples (V2)

Dimension	Metric	Score (1–5)	Focus Question
Reply Quality	Clarity	4.65	Is the reply professionally phrased, polite, and readable?
	Escalation Appropriateness	4.46	Does the reply avoid risky commitments when escalating?
	Relevance	4.32	Does the response directly address the customer's query?
	Overall Reply Score	3.56	Holistically, how effective is the customer response?
	Correctness	3.49	Are the technical troubleshooting recommendations sound?
Evidence Grounding	Helpfulness	3.49	Does the customer receive an immediate, actionable path forward?
	Unsupported Claims	3.69	Does the reply invent facts/policies not in evidence? (5 = None)
	Evidence Support	3.28	Are key factual assertions backed by retrieved evidence?
	Grounding Score	3.25	Overall degree to which the reply is anchored in evidence?
	Evidence Relevance	2.75	Did retrieval surface relevant historical cases?
Binary Decision	Grounding Supported %	81.02%	Proportion of replies with grounding_score >= 3 (175 / 216)

CAUTION: **Methodological Honesty on Model-Based Judging:** These scores reflect an automated LLM evaluator. A formal human-judge agreement study (Cohen's kappa) has **not yet been established** due to single-annotator constraints. While LLM-as-a-judge provides comparative visibility across prompt iterations, it cannot replace verified human operational review.

7. Controlled Experiment: V1 vs. V2 Reply Generator

In V2, we executed a controlled intervention on ReplyGenerator (src/agent/reply_generator.py): - **Intervention:** Prompts were stratified into three quality tiers (STRONG >= 0.70, RELEVANT 0.50 - 0.70, WEAK < 0.50) based on retrieval similarity. Internal tweet IDs and raw similarity numbers were stripped from prompt context, and the model was instructed to synthesize concrete resolution steps when evidence was strong, or specify missing diagnostic details when evidence was partial. Escalation policy was held strictly constant.

Experimental Comparison (V1 Full Judge vs. V2 Full Judge)

Metric Dimension	V1 Agent (N=215)	V2 Agent (N=216)	Observed Impact
Correctness	3.33 / 5	3.49 / 5	+0.16 (Improved)
Helpfulness	3.28 / 5	3.49 / 5	+0.21 (Improved)
Evidence Relevance	2.51 / 5	2.75 / 5	+0.24 (Improved)
Overall Reply Score	3.51 / 5	3.56 / 5	+0.05 (Modest Gain)
Clarity	4.68 / 5	4.65 / 5	-0.03 (Neutral)
Escalation Appropriateness	4.64 / 5	4.46 / 5	-0.18 (Regressed)
Grounding Score	3.35 / 5	3.25 / 5	-0.10 (Regressed)
Unsupported Claims Score	4.06 / 5	3.69 / 5	-0.37 (Regressed - More Hallucinations)
Binary Supported Rate	85.58%	81.02%	-4.56% (Regressed)

Critical Engineering Conclusion
V2 was NOT a universal improvement.

While prompt tiering successfully made the agent more proactive and helpful (+0.21 in helpfulness and +0.16 in correctness), it introduced a severe trade-off: **prompting the model to provide specific next steps when evidence was only moderately relevant caused it to hallucinate operational procedures** (unsupported claims dropped from 4.06 to 3.69). When the model was

discouraged from generic deferral, it filled evidential gaps by fabricating account lookups and warranty entitlement checks.

8. Top 5 Observed Failure Modes

Analyzing predictions and judge audits across the 216 Golden instances revealed five primary failure modes:

1. Fabricated Account/Order Verification & Entitlement Checks

- Mechanism:** When retrieved evidence lacked definitive resolution steps, V2's prompt instructed the model to state "what specific account information is required." The model overgeneralized, offering to perform account and order entitlement lookups for issues that were purely local OS bugs.
- Concrete Example (GOLDEN-0048):** Customer reported an OS update stall on a Nokia Lumia 930 phone.
Agent Reply: "Because basic troubleshooting has failed, please contact our support team so we can perform an order and account lookup..."
Judge Audit: Completely fabricated an order lookup requirement for a mobile operating system update failure.
- Concrete Example (GOLDEN-0004):** Customer asked why Windows 10 Fluent Design elements were missing.
Agent Reply: "This case requires additional investigation into your account entitlements and specifics..."

2. Conversation-State Blindness (Context Omission)

- Mechanism:** The agent classified and replied based heavily on the final customer turn, frequently ignoring entity mentions established in preceding conversation context.
- Concrete Example (GOLDEN-0042):** Customer asked: "What if I uninstall and then re-install the app?"
Preceding Context: Established that the user was troubleshooting contact synchronization in the Windows 10 **People App**.
Agent Reply: "To help us better understand your issue, could you please specify which application you are attempting to reinstall?"
Impact: Unnecessary friction; customer is forced to repeat context already provided.

3. Generic Deferrals Under Weak Retrieval

- Mechanism:** When retrieval returned low cosine similarity scores (< 0.50), the agent fell back to polite corporate deflection without addressing the customer's specific error code.
- Concrete Example (GOLDEN-0028):** Customer requested updated PowerShell/GUI instructions for modifying network adapter metric priorities in Windows 10. Retrieval surfaced unrelated generic Wi-Fi disconnect tweets. The agent replied with a boilerplate apology and a generic link to Windows Settings.

4. Premature & Rule-Induced Over-Escalation

- Mechanism:** Deterministic escalation rules treated entire intent categories (Complaint / Feedback, Billing & Payments) as mandatory human escalations regardless of query simplicity, yielding a low escalation precision of 23.68%.
- Concrete Example (GOLDEN-0020):** Customer expressed mild frustration about phone wait times while asking a basic settings question. The agent immediately triggered a hard escalation under the Complaint / Feedback intent rule, when a grounded self-service troubleshooting step would have defused the ticket.

5. Hallucinating Customer Problem Boundaries

- Mechanism:** Semantic search matched on superficial keywords (e.g., "storage"), causing the generator to diagnose an entirely different hardware malfunction.
- Concrete Example (GOLDEN-0039):** Customer asked: "Hey let me delete unnecessary windows apps I don't use on my surface pro 4 I want the storage back."
Agent Reply: Addressed the customer as though their Surface Pro SSD was corrupt and required hardware diagnostic replacement.

9. "What Is Misleading About My Headline Number?"

In AI engineering, presenting headline numbers without contextual caveats is unacceptable. We explicitly highlight the limitations and potential misinterpretations of our reported metrics:

- 40.28% Intent Accuracy is NOT Production-Ready:** In an enterprise production setting, a 40.28% multi-class accuracy means 6 out of every 10 incoming customer queries are assigned the wrong intent category. While it outperforms random guessing (10%) and exhibits genuine semantic spread over trivial baselines, it is insufficient for autonomous routing without human oversight.
- The 45.45% Baseline Paradox:** The naive majority baseline scored **45.45% accuracy**—higher than our AI Agent (40.28%). An uncritical stakeholder might conclude the majority baseline is superior. However, the majority baseline has a Macro F1 of only 6.94% and an Escalation F1 of 0.00%, achieving high accuracy purely by predicting Technical Troubleshooting for every ticket. Our agent achieves a Macro F1 of 28.80%, demonstrating real cross-category classification.
- Escalation Accuracy (63.43%) Masks Severe False-Positive Over-Escalation:** Escalation accuracy of 63.43% appears solid, but precision on ESCALATE_TO_HUMAN is only **23.68%**. The agent escalates 76 cases, but human reviewers labeled only 39 cases as requiring escalation. Over 75% of escalated tickets are false alarms, which would overwhelm human support queues in production.
- Golden Set Size & Extreme Class Imbalance:** The Golden Set contains 216 examples. While sufficient for disciplined error analysis, several classes have minimal sample counts: Microsoft Store (N=2), Warranty / Repair (N=4), Cancellation / Subscription (N=4). F1 scores for these minority classes exhibit wide statistical variance.

- Retrieval is the True Performance Ceiling:** Average evidence relevance scored only **2.75 / 5**. Because public tweets are terse (under 280 characters), dense embedding similarity frequently surfaces topically related but non-actionable tweets. The generative model cannot ground its replies when retrieved evidence lacks exact operational solutions.
- LLM-as-a-Judge Scores Overstate Real-World Reliability:** An 81.02% "Supported" rate from Gemini 2.0 Flash reflects an automated evaluator assessing another Gemini model. LLMs exhibit well-documented self-preference biases. True production grounding cannot be declared without verified inter-annotator agreement against human enterprise auditors.

10. What We Built Well

- End-to-End Runnable Pipeline:** Fully integrated, modular Python codebase connecting intent classification, vector search, tiered reply generation, and deterministic escalation into an operational CLI and Streamlit web application.
- Leakage-Free Historical Grounding:** Reconstructed 4,458 multi-turn conversation graphs from raw TWCS data. Built a FAISS IndexFlatIP dense vector store with strict exclusion of all Golden Set conversations, ensuring clean, uncontaminated benchmarking.
- Auditability & Explainable Escalation:** The escalation engine does not rely on opaque LLM confidence; it applies explicit, rule-based policies and outputs auditable structured rationale strings (predicted_escalation_reason).
- Comprehensive Test Suite:** 483 automated unit and integration tests covering data reconstruction, schema validation, FAISS edge cases, tiered prompt generation, and mock API resilience.
- Reproducible Evaluation Framework:** Caching engine using SHA-256 input fingerprinting prevents redundant API costs, supports seamless interrupt-and-resume workflows, and eliminates fabricated metrics.

11. What We Intentionally Did NOT Build

To maintain engineering discipline within the project scope, the following were intentionally excluded: 1. **Live Production Infrastructure:** No multi-node Kubernetes deployments, VPC peering, Kafka streaming queues, or autoscaling API gateways. 2. **Live Microsoft Backend Integrations:** No active OAuth connections to Microsoft Graph API, Azure Active Directory, Dynamics 365, or live billing/order systems. All order lookups and account checks are simulated conceptually. 3. **Foundation Model Fine-Tuning:** No parameter updates (PEFT/LoRA) on the foundation model. The system relies strictly on in-context learning and zero-shot prompt engineering. 4. **Full 3-Million-Tweet Indexing:** The FAISS index is intentionally scoped to the 4,458 reconstructed @MicrosoftHelps conversations rather than indexing the entire heterogeneous 2.8M TWCS dataset, optimizing for domain precision and memory footprint. 5. **CRM / Helpdesk Inbox Sync:** No direct two-way connectors to Zendesk, Freshdesk, or Hiver shared inboxes beyond the local Streamlit demonstration interface.

12. Next Week: Concrete Engineering Roadmap

Based on the top 5 failure modes, our next sprint focuses on targeted architectural upgrades:

NEXT SPRINT ROADMAP	
1. HYBRID RETRIEVAL & RERANKING	
Sparse BM25 (Exact Error Codes) + Dense Bi-Encoder + Cross-Encoder Reranker	
2. RESOLUTION-SPECIFIC INDEXING	
Index verified agent resolution turns rather than customer initial problem posts	
3. STRUCTURED CONVERSATION STATE TRACKING	
Extract & track {OS_Version, App_Name, Error_Code, Attempted_Steps} across turns	
4. POST-GENERATION CLAIM EXTRACTOR & CITATION FILTER	
Parse factual claims and drop replies containing ungrounded account/order lookups	
5. CALIBRATED PROBABILISTIC ESCALATION	
Replace binary intent blocks with a logistic risk model to reduce false escalations	
6. MULTI-ANNOTATOR HUMAN AGREEMENT	
Annotate 50 Golden Set replies with 2 independent human raters to compute Kappa	

- Hybrid Retrieval (Dense + Sparse BM25) with Cross-Encoder Reranker:** Dense semantic embeddings fail on exact alphanumeric error strings (e.g., 0x80070005, KB4048955). Implementing BM25 alongside all-MiniLM-L6-v2 with a cross-encoder/ms-marco-MiniLM-L-6-v2 reranker will elevate evidence relevance from 2.75 to > 3.8.

2. **Index Resolution Turns Specifically:** Currently, conversations are indexed by the customer's initial problem description. We will re-index specifically on the **agent's successful resolution turns**, ensuring retrieved evidence directly contains troubleshooting steps.
3. **Structured Dialogue State Tracking:** Implement an explicit state manager that tracks extracted slot entities ({device: "Sur-face Pro 4", os: "Windows 10", app: "People App"}) across turns, eliminating conversation-state blindness (GOLDEN-0042).
4. **Post-Generation Claim Extractor & Grounding Guardrail:** Introduce a lightweight post-generation verification step that parses generated sentences

- and vetoes any reply containing phrases like "account lookup" or "order check" unless explicitly authorized by the retrieved evidence snippet.
5. **Escalation Policy Calibration:** Replace blanket intent-level escalation triggers with a multi-factor risk model incorporating sentiment, customer tenure, and retrieval confidence, driving escalation precision from 23.68% toward > 60%.
6. **Establish Human-Judge Agreement:** Double-annotate 50 Golden Set instances across two human annotators to establish inter-rater reliability (Cohen's kappa), calibrating the automated LLM judge against human consensus.

13. Decision Log

#	Technical Decision	Selected Approach	Alternatives Considered	Engineering Justification
1	Target Brand Selection	Focus strictly on @MicrosoftHelps	AmazonHelp, AppleSupport, Uber_Support	Highest conversation depth (5.49 msgs/conv), 80.8% actionable guidance, and lowest DM deflection rate (17.5%).
2	Multi-Turn Graph Traversal	Recursive parent-tweet ID reconstruction	Treating each tweet as isolated flat document	Support interactions are non-Markovian; resolving queries requires preceding conversational state.
3	10-Intent Taxonomy Design	Mutually exclusive 10-class enterprise taxonomy	3 broad classes or 30+ granular classes	Avoided oversimplification while preventing catastrophic fragmentation on small evaluation subsets.
4	Dense Vector Indexing	FAISS IndexFlatIP with normalized vectors	FAISS IndexIVFFlat, ScaNN, ChromaDB	Normalized inner product provides exact, deterministic cosine similarity with zero quantization error on 4.4k items.
5	Leakage Prevention	Explicit holdout of 216 Golden IDs from FAISS index	Random train/test split of tweets	Tweet-level splitting leaks conversation turns into retrieval; holdout guarantees strict conversational isolation.
6	Human Golden Ground Truth	100% human-verified labels with AI assistance	Pure LLM synthetic labels	Eliminates model evaluation circularity; AI suggestions accelerated human labeling without polluting ground truth.
7	Deterministic Escalation	Explicit rule-based policy engine	End-to-end LLM escalation prompt	Enterprise compliance requires predictable, auditable escalation triggers for billing and account security.
8	Controlled V1 vs V2 Versioning	Isolated prompt tiering experiment	Concurrently altering retriever, model, and prompt	Isolates the independent variable (prompt tiering) to rigorously measure impact on grounding and hallucinations.
9	Inference Caching Architecture	SHA-256 input fingerprinting on local disk	In-memory cache or Redis database	Enables resumable evaluation runs without redundant Gemini API calls; auto-invalidates stale entries upon prompt edits.
10	Scoped Dataset Indexing	Reconstructing only @MicrosoftHelps (4,458 convs)	Indexing all 2.8M TWCS records	Drastically reduced embedding time from hours to minutes while eliminating cross-domain retrieval noise (e.g., airline tweets).
11	Immutable Evaluation Artifacts	Freezing V1/V2 predictions and evaluation reports	Overwriting predictions.json in place	Guarantees scientific auditability, reproducibility, and prevents accidental data contamination.
12	Decoupled Evaluation Engine	Strict separation of inference runner from evaluator	Calculating metrics inside inference scripts	Prevents model inference from accessing ground-truth labels, guaranteeing zero operational leakage.
13	Strict Pydantic Validation	Enforcing JSON schema parsing with error logs	Loose regex or free-form text extraction	Catches schema deviations (e.g., GOLDEN-0157) and records failures deterministically in predictions_failures.json.
14	Dual Baseline Architecture	Majority Class + TF-IDF Logistic Regression	Fine-tuned BERT or zero baseline	Demonstrates that high nominal accuracy (79.5%) can be achieved naively without solving the underlying task.

14. Reproduction Guide (Under 15 Minutes)

Follow these steps to set up the environment, run tests, and reproduce the evaluation results.

Step 1: Environment Setup & Dependencies (2 minutes)

```
# Clone repository and navigate to root
cd d:\WEB\project\hiver-ai-agent

# Create and activate virtual environment
python -m venv venv
.\venv\Scripts\Activate.ps1

# Install required dependencies
pip install streamlit pydantic google-genai `
scikit-learn sentence-transformers faiss-cpu pytest
```

Step 2: Configure Environment Variables (1 minute)

```
# Set Gemini API key and optional model configuration
$env:GEMINI_API_KEY = "your-gemini-api-key-here"
$env:GEMINI_MODEL = "gemini-2.0-flash"
$env:AGENT_EMBEDDING_INDEX = "evaluation/twcs_evidence_index.json"
```

Step 3: Run Full Automated Test Suite (1 minute)

Verify pipeline integrity across all 483 unit and integration tests:

```
pytest
```

Expected result: 483 passed in ~30s.

Step 4: Reproduce Baseline Metrics (1 minute)

Compute the Majority and TF-IDF + Logistic Regression baselines:

```
python -m evaluation.run_baselines
```

Output generated: evaluation/baseline_results/report.md confirming 45.45% majority baseline accuracy.

Step 5: Reproduce V2 Evaluation Metrics (Instant from Frozen Predictions)

Verify the frozen V2 automated evaluation metrics without making external API calls:

```
python -m evaluation.evaluate `
--predictions evaluation/predictions_v2.json `
--golden-set evaluation/golden_set.json
```

Expected result: - Total Evaluated: 216 - Intent Accuracy: **40.28%** - Intent Macro F1: **28.80%** - Escalation Accuracy: **63.43%** - Escalation F1: **31.30%**

Step 6: Launch Interactive Streamlit Application (Optional Demo)

Test the live agent with interactive multi-turn inputs:

```
streamlit run app.py
```

Opens web browser at http://localhost:8501 demonstrating live classification, FAISS retrieval, and grounded reply generation.

15. References & Borrowed Work

- TWCS Dataset:** Kaggle Customer Support on Twitter Dataset (twcs.csv), comprising 2.8M customer support tweets. Reference: *Customer Support on Twitter*, Kaggle (2017).
- Dense Sentence Embeddings:** sentence-transformers/all-MiniLM-L6-v2, mapping sentences to a 384-dimensional dense vector space. Reimers & Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks", EMNLP 2019.
- Similarity Search:** FAISS (Facebook AI Similarity Search) library for high-performance dense vector indexing and cosine inner product computation (IndexFlatIP). Johnson et al., "Billion-scale similarity search with GPUs", IEEE Transactions on Big Data (2019).
- Foundation LLM:** Google Gemini 2.0 Flash (gemini-2.0-flash) via google-genai Python SDK for zero-shot intent classification, grounded synthesis, and automated LLM-as-a-judge scoring.
- Machine Learning & Evaluation:** Scikit-Learn library for TF-IDF vectorization, Logistic Regression baselines, multi-class confusion matrices, and Macro/Weighted F1 score calculations. Pedregosa et al., JMLR 2011.
- Data Validation & UI:** Pydantic v2 for structured JSON parsing and schema validation; Streamlit for interactive prototyping and auditor demonstration.